

Model Comparison for Yelp Sentiment Analysis

Table of Contents

Table of Contents	2
Introduction	3
Dataset Description and Processing	4
Common Preprocessing Workflow	4
DistilBERT Specific Workflow	4
LSTM Specific Workflow	4
Model Descriptions	6
DistilBERT	6
LSTM Architecture & Hyperparameter Choices	6
Results and Findings	7
DistilBERT	7
Hyperparameter Tuning.....	7
Optimized Model Evaluation	8
Interpretability Analysis with LIME.....	9
Vocabulary Analysis (Bag of Words + Word Cloud)	9
LSTM.....	10
Hyperparameter Tuning.....	10
Optimized Model Evaluation	11
Vocabulary Analysis (Bag of Words + Word Cloud)	14
Interpretability Analysis with LIME.....	15
Model Comparison (tuned)	17
Analysis and Discussion	18
Conclusion	19
References	20

Introduction

This project trains and compares two deep learning models, LSTM and DistilBERT, to complete sentiment analysis for Yelp reviews of restaurants or hotels (inclusive). The goal is to compare the strengths and weaknesses of the models in their ability to utilize the subjective opinions in the reviews to determine the sentiment (Negative, Neutral, or Positive). Sentiment analysis is useful because reviews contain direct customer feedback, but they are too numerous to read manually. A sentiment model can help summarize general customer opinion and identify common strengths or complaints. Furthermore, a comparison analysis between two models can help determine which model to select for similar studies in the future.

Dataset Description and Processing

The original data came from the Yelp Open Dataset, an educational dataset containing real business information and review text [1]. This study focuses on the review.json and business.json data files within the broader Dataset, which comprises of the following relevant fields and value types:

Data File	Attribute	Type	Purpose
review.json	'business_id'	String	Unique id of the business being reviewed (for matching)
	'stars'	int	Rating of the business, from 1 (lowest) to 5 (highest)
	'text'	String	Customer's textual explanation of their review/experience.
business.json	'business_id'	String	Unique id of the business being reviewed (for matching)
	'categories'	String	Industries this business belongs to ('hotel', 'restaurants')

Common Preprocessing Workflow

involved selecting businesses that contain the 'hotel' or 'restaurant' category, retrieving the stars and text from reviews for these businesses, converting stars into three sentiment classes (1, 2 → Negative, 3 → Neutral, 4, 5 → Positive), and balancing the class representations by downsampling all classes based on the minority class.

DistilBERT Specific Workflow

A balanced sample of 99,900 reviews was used: 33,300 reviews per sentiment class. Balancing was important because it prevented the model from receiving more examples from one sentiment than another. The sample was then split into training, validation, and test sets.

Dataset split	Reviews	Purpose
Training	71,928	Model Training
Validation	7,992	Hyperparameter Tuning
Test	19,980	Final unbiased evaluation

Reviews were tokenized with the DistilBERT tokenizer. A maximum length of 256 tokens was used so that the model could process enough context while keeping GPU memory and training time manageable. Labels were encoded as 0 for Negative, 1 for Neutral, and 2 for Positive.

LSTM Specific Workflow

LSTM specific preprocessing involved data splitting with stratification of classes across all sets, data sanitization (irrelevant characters) while maintaining negation stopwords, tokenization using spacy to build a vocabulary of 20,000 words (with reserved <pad> and <unk> entries), token mapping to a 100-dimensional pre-trained GloVe vector, conversion of text to sequences,

padding, and then conversion to PyTorch Tensors and Dataloaders. The detailed splits and sample sizes are shown below

Dataset Split & Sample Sizes

Dataset split	Reviews	Purpose
Training	72,000	Model Training
Validation	8,000	Hyperparameter Tuning
Test	20,000	Final Model Evaluation

Sentiment	Train	Validation	Test
Negative	23856	2622	6767
Neutral	23966	2660	6656
Positive	24178	2718	6577

Model Descriptions

DistilBERT

DistilBERT is a smaller version of BERT created through knowledge distillation. It keeps much of BERT's language understanding while requiring less computation, which makes it suitable for a student project with limited training resources [2]. The pre-trained distilbert-base-uncased model was fine-tuned for a three-class text-classification task.

The main fixed settings were a maximum sequence length of 256 tokens, a batch size of 16 during the baseline configuration, a learning rate of $2e-5$, and up to 10 epochs. Early stopping was enabled, so training stopped when validation performance no longer improved. This reduced unnecessary training and helped limit overfitting.

LSTM Architecture & Hyperparameter Choices

The **LSTM** is a recurrent, sequence-based model that leverages multiple gates to manage short-term and long-term information. For the one built for this project, the LSTM processes a review one token at a time and carries a hidden state forward while preserving word order [6]. The network stacks an embedding layer initialised with GloVe vectors, a bidirectional two-layer LSTM, and a small fully connected classification head. Reading the sequence in both directions lets the model use context from either side of a word, and the final hidden states from the two directions are concatenated before the head.

Regularization is applied in three places: the pre-trained embeddings are frozen, dropout is applied between the stacked LSTM layers, and dropout is applied before every linear layer in the head. The settings below are the tuned configuration.

Component	Setting
Embedding layer	20,000 x 100, GloVe-initialised, frozen
LSTM	128 hidden units per direction, 2 layers, bidirectional
LSTM dropout	0.26 (between stacked layers)
Fully connected head	2 hidden layers of 64 units with ReLU
Head dropout	0.19 (before every linear layer)
Output layer	3 classes (Negative, Neutral, Positive)
Optimizer	AdamW, learning rate 0.0041, weight decay $2e-5$
Batch size / epochs	256 / 10

Results and Findings

DistilBERT

Hyperparameter Tuning

Hyperparameter tuning tested six simple alternatives. Only the validation set was used to select the final configuration. The best run used a learning rate of $1e-5$, a batch size of 16, and dropout of 0.2. It achieved the highest validation macro-F1 score of 0.8227.

Run	Learning rate	Batch	Dropout	Validation accuracy	Validation macro-F1
lr_1e-5	$1e-5$	16	0.2	0.8219	0.8227
baseline	$2e-5$	16	0.2	0.8187	0.8202
batch_32	$2e-5$	32	0.2	0.8202	0.8184
dropout_0.3	$2e-5$	16	0.3	0.8177	0.8183
dropout_0.1	$2e-5$	16	0.1	0.8159	0.8174
lr_3e-5	$3e-5$	16	0.2	0.8152	0.8162

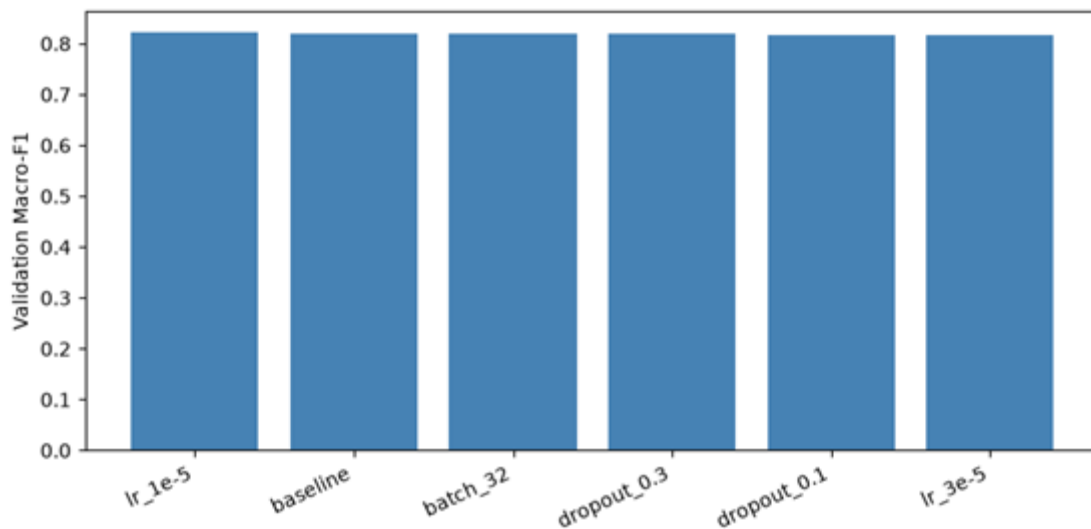


Figure 1. Validation macro-F1 across the six hyperparameter runs.

The results show that a lower learning rate of $1e-5$ gave the best validation result. Increasing the learning rate to $3e-5$ reduced macro-F1. Changing dropout or increasing the batch size did not improve the selected baseline in this experiment.

Optimized Model Evaluation

The selected model was evaluated once on the held-out test set. The final test macro-F1 was 0.8219, which is very close to the validation macro-F1 of 0.8227. This suggests that the chosen configuration performed consistently on unseen reviews.

Overall test metric	Result
Accuracy	0.8216
Macro precision	0.8223
Macro recall	0.8216
Macro-F1	0.8219

The model was strongest on Positive reviews and weakest on Neutral reviews. This is understandable because neutral reviews often contain mixed opinions, while positive and negative reviews more frequently use clear sentiment words.

Class	Precision	Recall	F1-score
Negative	0.8500	0.8374	0.8437
Neutral	0.7354	0.7486	0.7420
Positive	0.8815	0.8787	0.8801

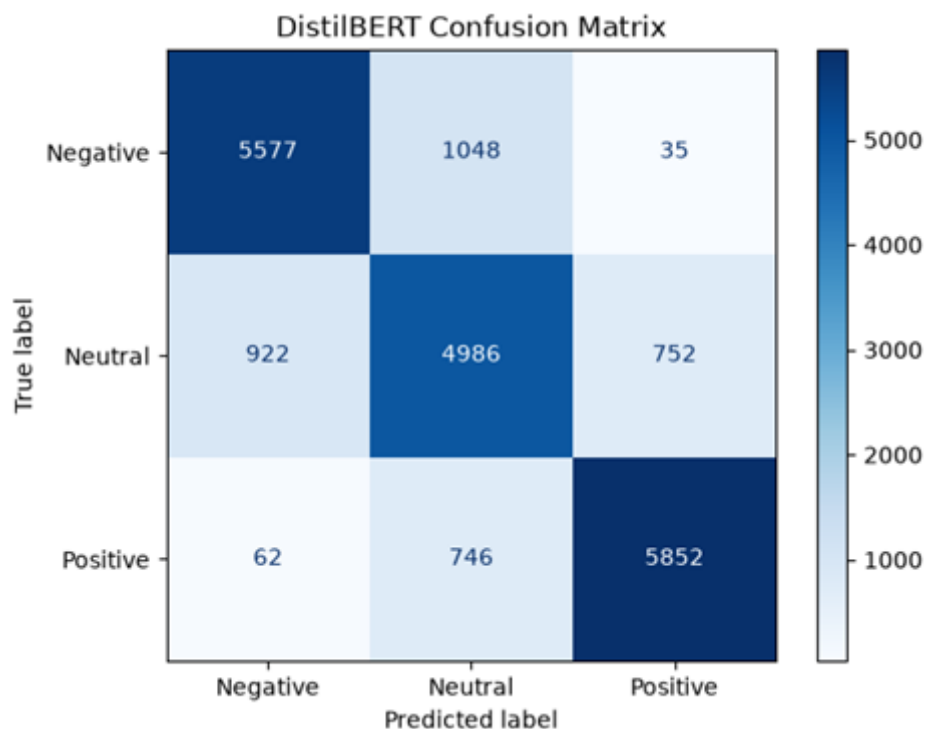


Figure 2. Confusion matrix for the final DistilBERT model on the test set.

The confusion matrix has a strong diagonal, meaning that most reviews were classified correctly. The largest remaining challenge is distinguishing neutral reviews from nearby positive or negative opinions.

Interpretability Analysis with LIME

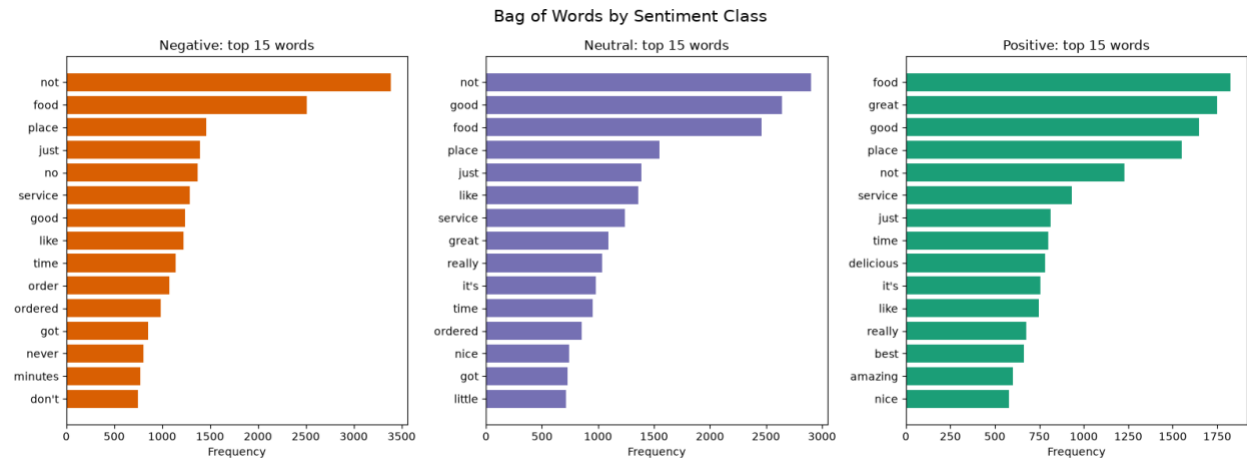
LIME was used to explain individual predictions. LIME creates a simple local explanation around one prediction and shows which words pushed the model toward a class [3]. It does not describe the entire model; instead, it helps explain a specific review.

For a correctly classified negative review, terms such as “don’t,” “told,” “weeks,” and “disappointed” supported the Negative prediction. For a correctly classified positive review, “delicious,” “great,” and “worth” strongly supported the Positive prediction. The neutral example contained conflicting words such as “delicious,” “stale,” and “disappointment,” which illustrates why neutral sentiment is harder: a review can contain both positive and negative evidence.

One error involved a Neutral review predicted as Positive. This example shows an important limitation: LIME can reveal which words influenced the prediction, but an influential word is not always intuitively positive or negative by itself. Its contribution depends on the context and the patterns learned by the model. Therefore, LIME was used here as a transparent local diagnostic rather than as a complete explanation of all model behavior.

Vocabulary Analysis (Bag of Words + Word Cloud)

To obtain an initial overview of the language used in the app reviews, a Bag of Words analysis and a Word Cloud were generated from the preprocessed review corpus. The Bag of Words chart identifies the terms that occur most frequently across the dataset, while the Word Cloud provides a complementary visual representation in which more frequent words appear in a larger size. Together, these visualizations help describe the dominant vocabulary before applying supervised sentiment-classification models.



The results suggest that the reviews are strongly focused on the users' experience with the banking application and its services. Recurrent vocabulary is associated with practical aspects of app usage, such as access, account management, transactions, support, and service quality. This pattern is expected because users typically write reviews to report a problem, request an improvement, or express satisfaction with a specific feature. However, word frequency alone does not determine sentiment: the same high-frequency term may occur in both positive and negative reviews, depending on the surrounding context. For this reason, the vocabulary analysis is used as an exploratory step, whereas the final sentiment predictions rely on the TF-IDF and transformer-based models, which capture a broader representation of the review text.



LSTM

Hyperparameter Tuning

Tuning used Optuna with a TPE sampler over eleven hyperparameters, maximising validation macro-F1 [7]. Ten trials were run; a median pruner stopped unpromising trials early, leaving six completed and four pruned. The baseline configuration was enqueued as trial 0 so that every other result is measured against a fixed reference. Only the validation split was used for selection.

Trial	Learning rate	LSTM layers	Bidirectional	LSTM dropout	Validation macro-F1
3 (best)	0.0041	2	Yes	0.26	0.7706
2	0.00026	2	Yes	0.19	0.7585
0 (baseline)	0.0010	1	No	0.20	0.7419
1	0.0067	2	No	0.10	0.7348
5	0.0011	1	No	0.17	0.7181
6	0.0073	1	Yes	0.14	0.7141

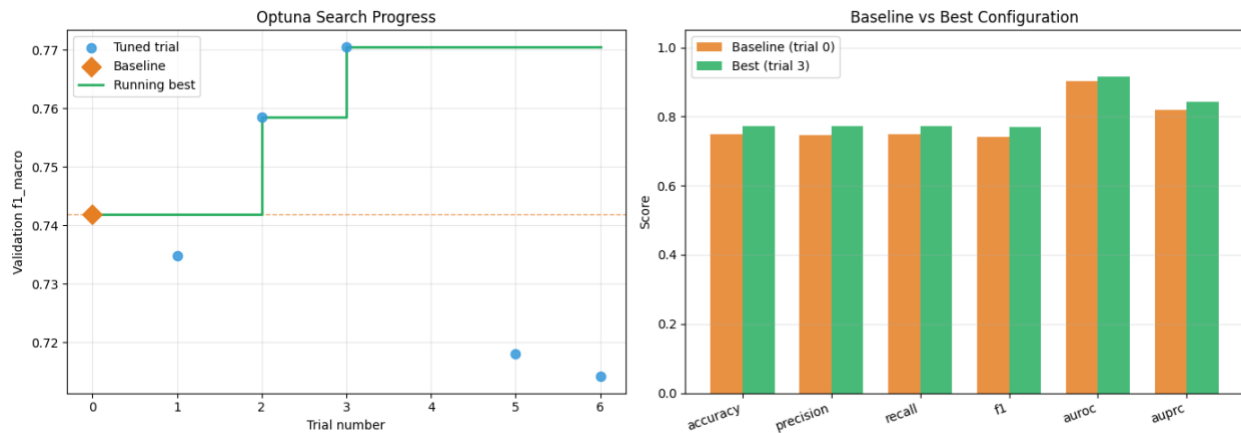


Figure 3. Optuna search progress and the baseline-versus-best comparison across validation metrics for the LSTM.

The two strongest trials were both two-layer bidirectional networks, which suggests that depth and bidirectionality mattered more than the learning rate on its own. Trial 3 raised validation macro-F1 to 0.7706 from the baseline 0.7419, a gain of 2.9 points. The three weakest completed trials all unfroze the embedding layer, which let the model overwrite the pre-trained GloVe vectors and cost several points of macro-F1.

Optimized Model Evaluation

The tuned configuration was retrained and then evaluated once on the 20,000-review held-out test set. The weights from the best validation epoch were restored rather than those from the last epoch.

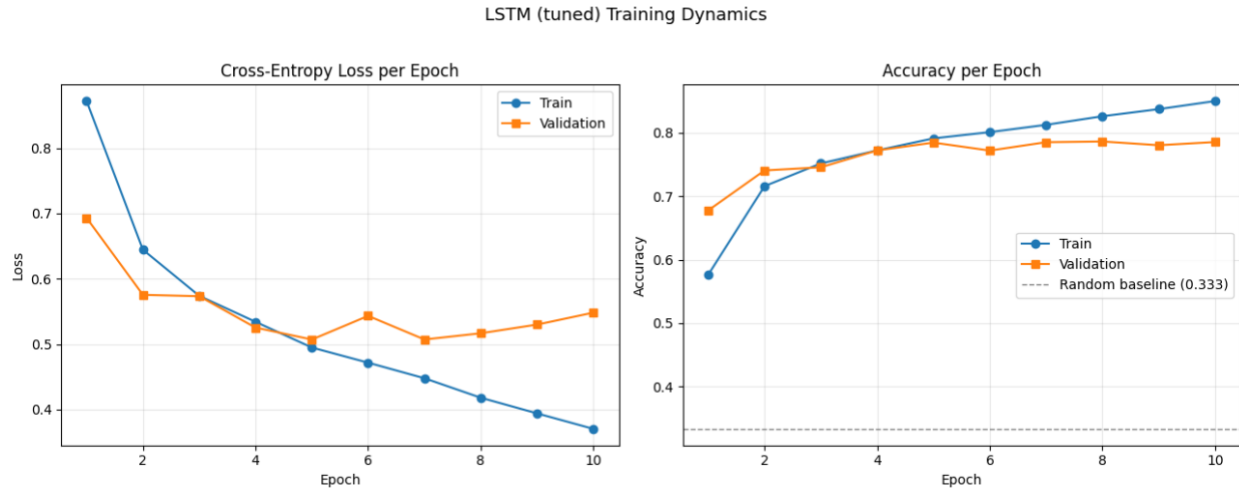


Figure 4. LSTM training and validation loss and accuracy per epoch.

Training accuracy reached 0.8501 while validation accuracy plateaued at 0.7864 at epoch 8, a gap of 6.5 points. This indicates mild overfitting in the later epochs despite dropout and frozen embeddings, and it is the reason the best-epoch weights were kept.

Overall test metric	Result
Accuracy	0.7840
Macro precision	0.7945
Macro recall	0.7844
Macro-F1	0.7866

As with DistilBERT, the LSTM was strongest on Positive reviews and weakest on Neutral ones.

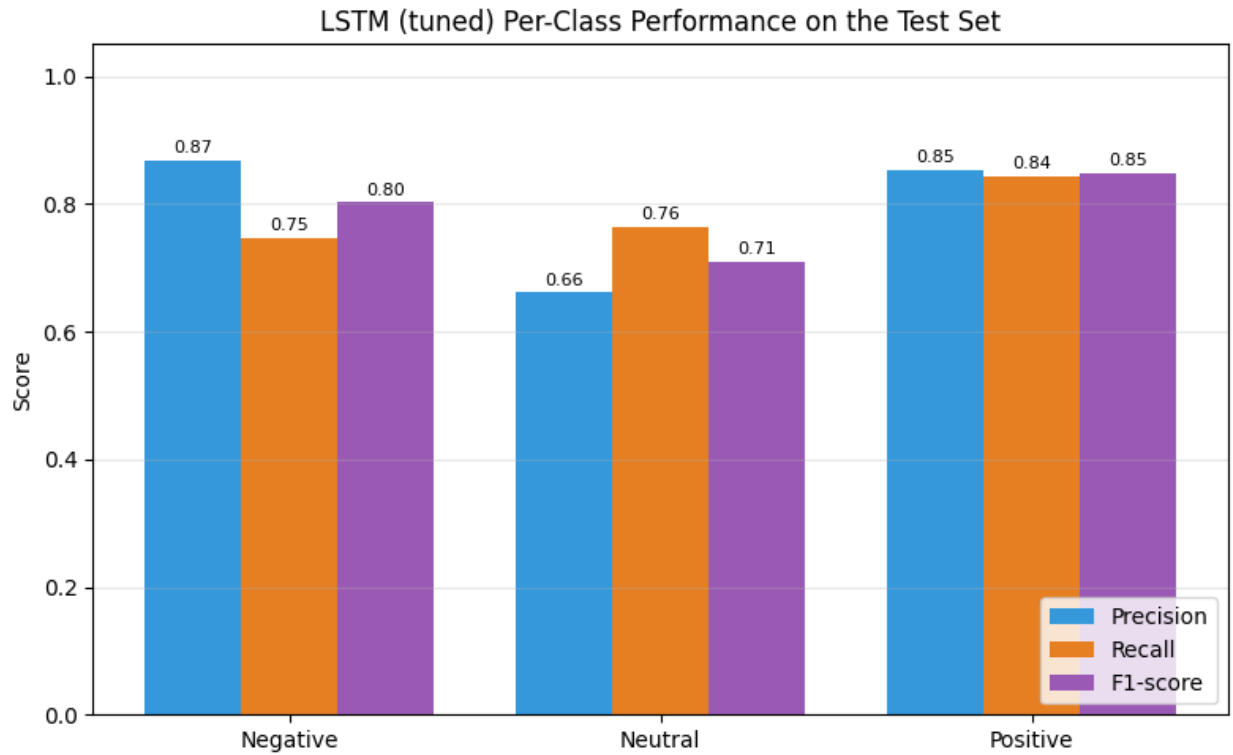


Figure 5. Bar Chart for tuned LSTM model to Compare Precision, Recall, F1-Score by Sentiment

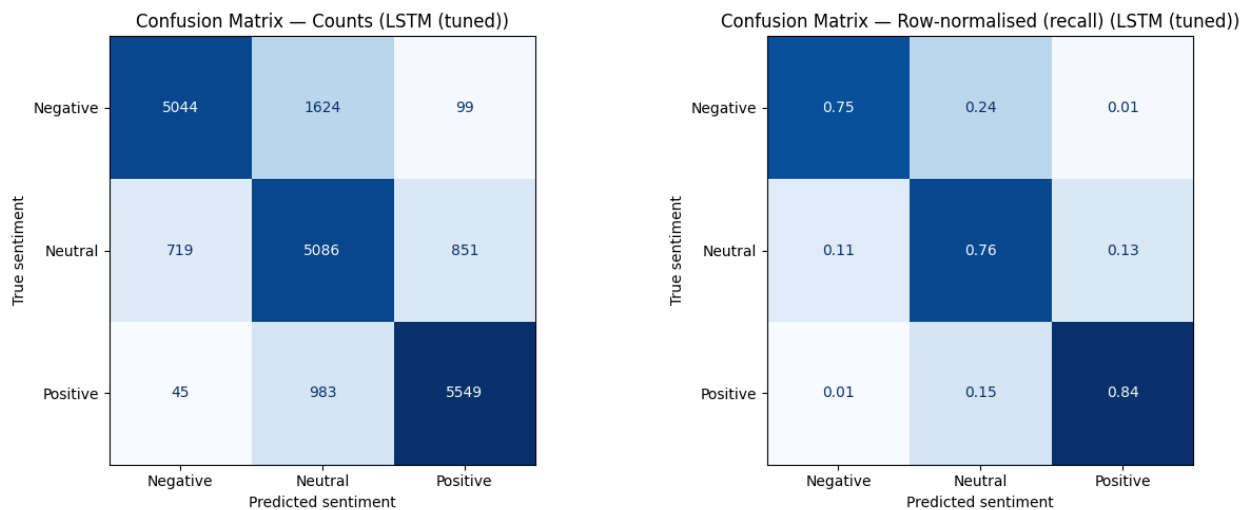


Figure 6. Confusion matrices for the final LSTM model on the test set, as raw counts and normalised by row.

Negative recall is the weakest result at 0.745, with 24.0% of negative reviews labelled Neutral. Positive reviews were handled best, at 0.844 recall. Neutral precision is only 0.661, meaning the class absorbs errors from both of its neighbours: it sits between the other two classes on a sentiment scale, so any uncertainty tends to land there.

The table below tracks the same metrics across the three evaluation stages.

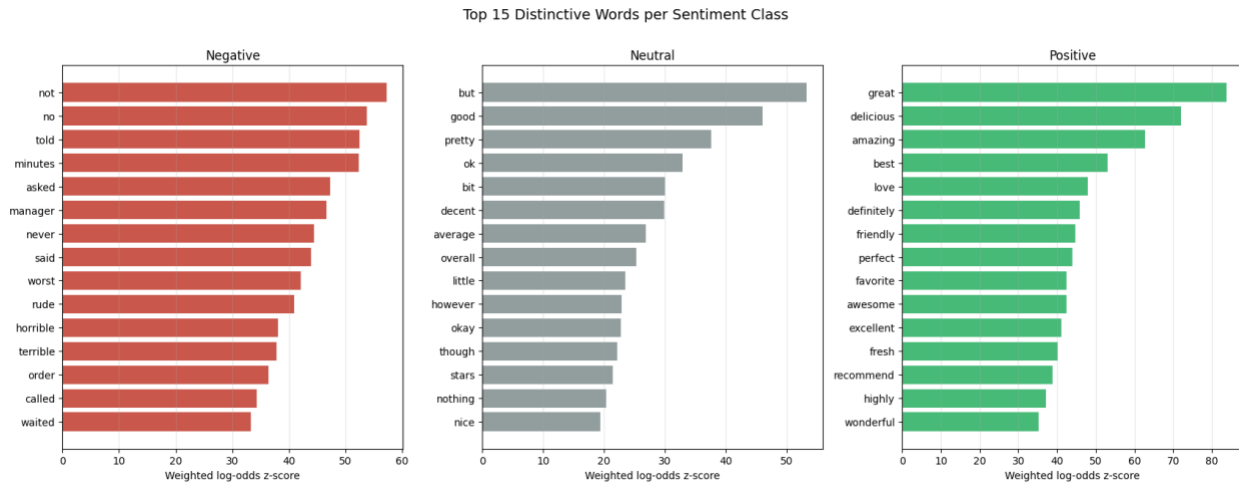


Figure 9. Weight log-odds score horizontal bar chart to represent top 15 Distinct words per sentiment class

Interpretability Analysis with LIME

LIME was applied to the LSTM through a wrapper that reproduces the full inference path, from cleaning through to the softmax. The wrapper was checked against the stored test predictions on 200 reviews and agreed on all of them, so the explanations describe the model that was actually evaluated. Explanations were then aggregated over a stratified sample of 90 test reviews, 30 per class, with 800 perturbations each.

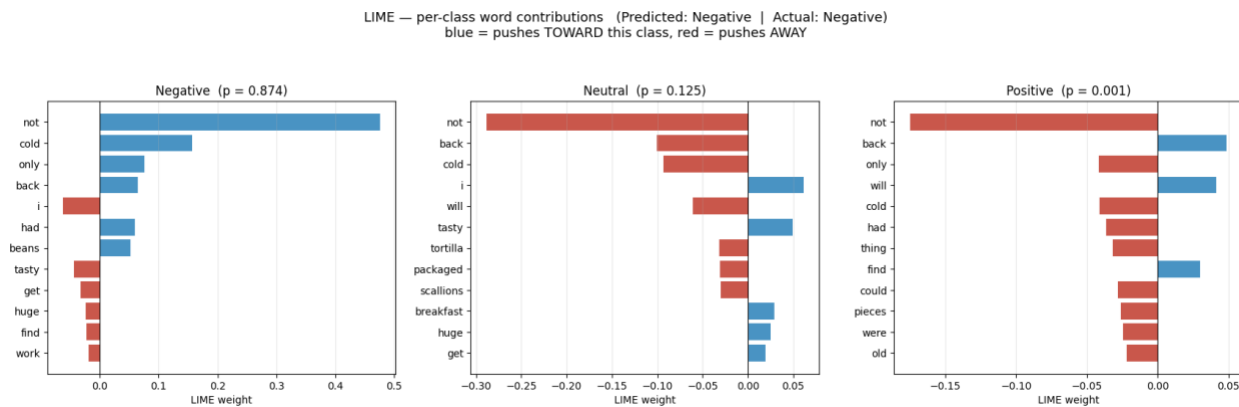


Figure 10. LIME word contributions for a correctly classified negative review. Blue pushes toward the class, red pushes away.

Aggregated over the sample, the words pushing most strongly toward each class were "no" (+0.13), "bad" (+0.09) and "not" (+0.08) for Negative; "but" (+0.09), "ok" (+0.07) and "pretty" (+0.05) for Neutral; and "amazing" (+0.17), "love" (+0.12) and "awesome" (+0.11) for Positive. The model therefore leans on negation and contrast tokens, not only on sentiment-bearing adjectives, which confirms that keeping those words out of the stop-word list was correct.

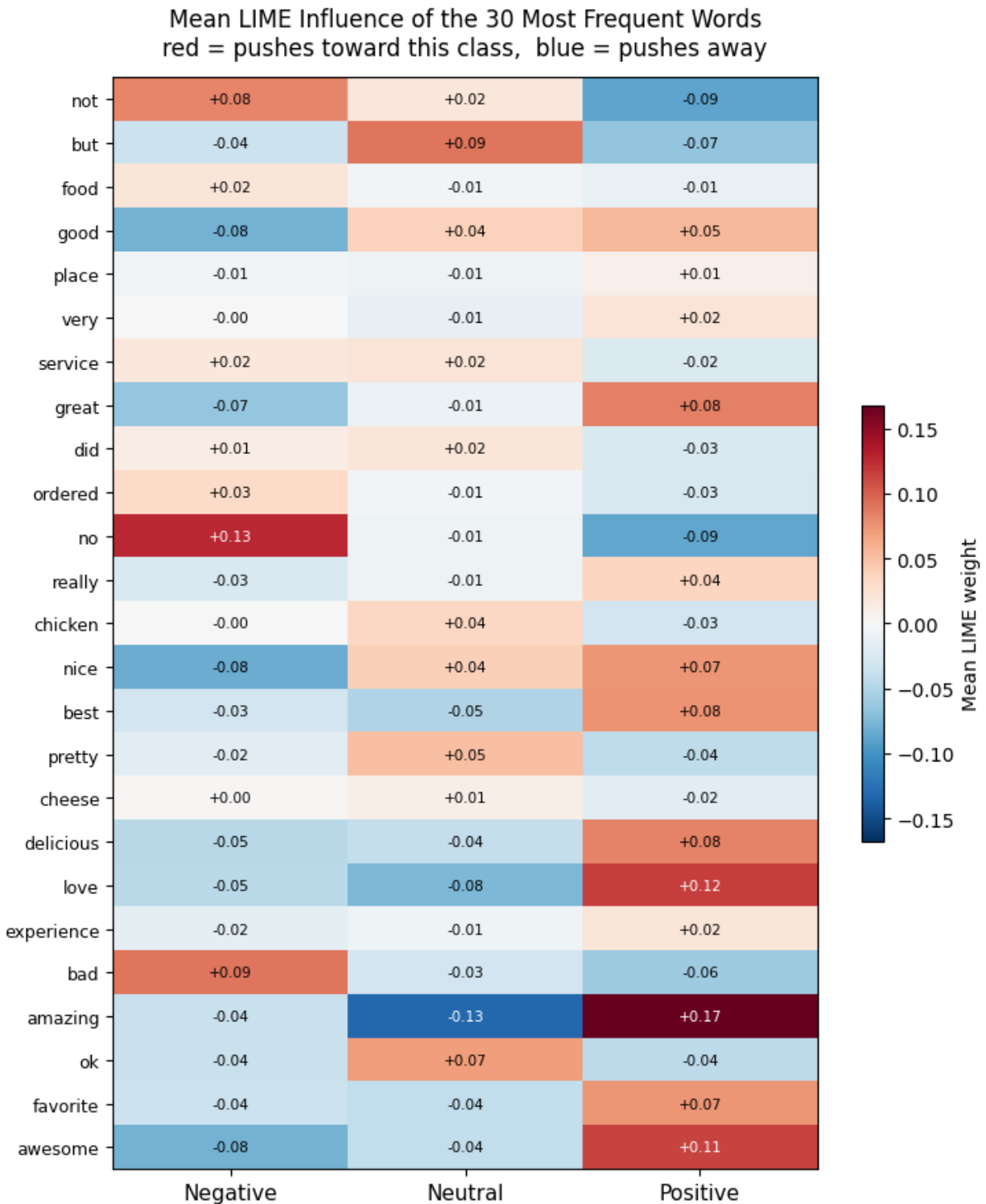


Figure 11. Mean LIME influence of the most frequent vocabulary words on each sentiment class.

The heatmap shows the same pattern across the most frequent vocabulary: "not" and "no" carry the largest weights toward Negative, "but" is the clearest Neutral marker, and the strongest Positive weights belong to a small set of intensifiers. This agrees with the log-odds ranking

above, so the model's decisions are consistent with the corpus statistics rather than driven by incidental vocabulary.

Model Comparison (tuned)

Both models were trained on the same balanced three-class Yelp sample and scored on a held-out test split of the same size and composition.

Metric	DistilBERT	LSTM
Accuracy	0.8216	0.7840
Macro precision	0.8223	0.7945
Macro recall	0.8216	0.7844
Macro-F1	0.8219	0.7866
Negative F1	0.8437	0.8022
Neutral F1	0.7420	0.7089
Positive F1	0.8801	0.8487

Analysis and Discussion

The final DistilBERT model achieved solid and balanced overall performance, with an accuracy of 82.16% and macro-F1 of 82.19%. Macro-F1 is important because it gives equal importance to Negative, Neutral, and Positive classes. The close validation and test scores indicate that the selected configuration generalized well to the held-out data.

The model recognized positive sentiment most effectively. Clear words such as “delicious,” “great,” and “worth” appeared in the LIME explanations and matched the model’s strong Positive F1-score. Negative performance was also strong, while Neutral performance was lower. This is the main area for improvement because neutral reviews often include mixed or less direct sentiment.

DistilBERT outperformed the LSTM on every metric, with a macro-F1 of 0.8219 against 0.7866, a gap of 3.5 points. The gap is spread evenly across all three classes rather than concentrated in one, which points to a general advantage in language representation rather than a class-specific effect. This is the expected outcome: DistilBERT brings contextual pre-training, whereas the LSTM reads static GloVe vectors that give a word the same representation regardless of the sentence it appears in.

The two models had similar strengths and weaknesses when it comes to classifying the three sentiments. Both found Neutral hardest (F1 0.7420 against 0.7089) and Positive easiest (0.8801 against 0.8487), and both sets of LIME explanations pointed to the same cause, namely that neutral reviews mix positive and negative evidence. The LSTM vocabulary analysis made this concrete: neutral reviews are identified by hedges and contrast words such as “but”, “ok” and “pretty” rather than by sentiment words of their own. Positive and negative sentiments are frequently misclassified as neutral since the neutral drive factors also contain a mixture of positive and negative sentiment words; reviewers providing a rating of 3 out of 5 stars often do so due to mixed sentiments, rather than lacking any sentiment.

Two caveats apply to the comparison. The two models were tuned with different search budgets and strategies, so the gap reflects the configurations reached rather than the best possible version of either architecture. The LSTM was also far more expensive to tune in wall-clock terms, taking roughly eleven hours for ten Optuna trials plus a further three and a half hours for the final fit. A wider LSTM search might narrow the gap, but it is unlikely to close it.

Conclusion

This project demonstrated that fine-tuned DistilBERT can classify Yelp review sentiment effectively across three balanced classes. Hyperparameter tuning selected a learning rate of $1e-5$, batch size of 16, and dropout of 0.2. The resulting model achieved 82.16% test accuracy and 82.19% macro-F1. The model handled Positive and Negative reviews well, but Neutral reviews remained more difficult. The LIME examples supported this conclusion by showing that neutral reviews can contain conflicting sentiment words.

The LSTM reached 78.40% test accuracy and 78.66% macro-F1. Optuna tuning raised validation macro-F1 from 0.7419 to 0.7706 over the baseline, and the best configuration was a two-layer bidirectional network with frozen GloVe embeddings. Its vocabulary and LIME analyses showed that the model relies on negation and contrast tokens as much as on sentiment adjectives.

Comparing the two architectures, DistilBERT was the stronger model by roughly 3.5 macro-F1 points. Both models performed best for Positive reviews and struggled with Neutral reviews; from the perspective of sentiment analysis, they had similar strengths and weaknesses, but the pre-trained DistilBERT model demonstrated superior results.

Future work to improve the project includes widening the hyperparameter tuning search space, increasing the number of HPO trials, trying alternative ways to process the data based on learnings from the vocabulary analysis, refining the neutral-label definition from the interpretability analysis, evaluating whether difference sequence lengths improve Neutral prediction and training the model with a larger sample size (beyond 100,000 total samples).

References

1. Yelp. Yelp Open Dataset. <https://business.yelp.com/data/resources/open-dataset/>
2. Sanh, V., Debut, L., Chaumond, J., and Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108. <https://arxiv.org/abs/1910.01108>
3. Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. arXiv:1602.04938. <https://arxiv.org/abs/1602.04938>
4. Hugging Face. Transformers Documentation. <https://huggingface.co/docs/transformers/>
5. J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global Vectors for Word Representation," Stanford NLP Group, Jul. 2025. [Online]. Available: <https://nlp.stanford.edu/projects/glove/>. [Accessed: Jul. 27, 2026].
6. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Comput., vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735. [Online]. Available: IEEE Xplore, <https://ieeexplore.ieee.org/abstract/document/6795963>. [Accessed: Jul. 27, 2026].
7. T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2019, pp. 2623–2631, doi: 10.1145/3292500.3330701.
8. A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems 32, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds. Red Hook, NY, USA: Curran Associates, Inc., 2019, pp. 8024–8035.